

INFORME TÉCNICO DE PROYECTO

Adquisición y Clasificación de Señales EMG

Sistema de control gestual mediante señales musculares — ESP32, AD8232 y Machine Learning

Jose Luis Bravo Bolaños
Ana María García Salazar

Universidad del Cauca

Bioingeniería · Ingeniería en Automática Industrial ·
2026

1. Introducción

El presente informe documenta de manera integral el desarrollo de un sistema de adquisición, preprocesamiento y clasificación de señales electromiográficas de superficie (sEMG), orientado al reconocimiento de gestos de la mano mediante hardware de bajo costo. El proyecto surge como iniciativa académica entre los programas de Bioingeniería e Ingeniería en Automática Industrial de la Universidad del Cauca, con la visión de sentar las bases tecnológicas para el control de una prótesis miolétrica.

La electromiografía de superficie consiste en la captación de los potenciales eléctricos generados por las fibras musculares durante su contracción. Cuando el sistema nervioso envía una orden motora, se generan potenciales de acción que pueden ser detectados por electrodos colocados sobre la piel, amplificados y digitalizados para su análisis computacional.

El principal desafío de este tipo de señales radica en su naturaleza estocástica y su baja amplitud —entre 50 μV y 5 mV—, lo que las hace extremadamente susceptibles al ruido eléctrico ambiental, a los artefactos de movimiento y al potencial de contacto electrodo-piel. Superar estos desafíos de forma sistemática constituyó el eje central del trabajo.

El documento está organizado siguiendo el flujo completo del proyecto: desde la descripción del hardware y el firmware del microcontrolador, hasta el pipeline de clasificación en Python y la interfaz gráfica final. Cada sección incluye no solo la explicación técnica de las decisiones de diseño, sino también las instrucciones precisas de ejecución, de modo que el documento sirva simultáneamente como informe técnico y como guía de reproducción del sistema.

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar un sistema que capture señales EMG del antebrazo, las procese mediante filtros digitales y clasifique al menos cuatro gestos distintos de la mano usando hardware de bajo costo y software libre, como punto de partida para el control de una prótesis.

2.2 Objetivos específicos

- OE1 — Protocolo de adquisición: definir la posición de los electrodos, la frecuencia de muestreo, el tiempo por gesto y las condiciones para reducir el ruido de medición.

- OE2 — Cadena de preprocesamiento: implementar filtro de promedio móvil, calibración automática de línea base y detección de electrodos sueltos directamente en el firmware del ESP32.
- OE3 — Extracción de características: calcular seis descriptores estadísticos por ventana de señal (MAV, RMS, ZCR, varianza, longitud de onda e IEMG).
- OE4 — Entrenamiento y evaluación: entrenar un clasificador SVM con kernel RBF y evaluar su exactitud mediante reporte de clasificación y matriz de confusión.
- OE5 — Interfaz en tiempo real: construir una interfaz gráfica interactiva que muestre la señal EMG y el gesto clasificado de extremo a extremo.

Al momento de este informe, los objetivos OE1 y OE2 han sido completados, y OE3 y OE4 se encuentran funcionales con una precisión global del 94.1%. El objetivo OE5 está parcialmente implementado mediante el juego del dinosaurio descrito en la Sección 6.

3. Hardware utilizado

3.1 Sensor AD8232 — Amplificador de señal biopotencial

El AD8232 es un circuito integrado de Analog Devices diseñado específicamente para la adquisición de señales biopotenciales débiles, como ECG y EMG. Es el componente central de la cadena de adquisición y resuelve de forma analógica los problemas fundamentales de la medición EMG superficial.

Internamente, el chip contiene un amplificador de instrumentación que amplifica únicamente la diferencia de potencial entre los dos electrodos de señal (IN+ e IN-), rechazando cualquier señal que aparezca de forma idéntica en ambas entradas. Este principio, denominado rechazo de modo común, permite cancelar la interferencia de la red eléctrica a 60 Hz antes de que llegue al ADC.

Adicionalmente, incorpora un filtro pasa-altas que elimina el voltaje de continua del contacto electrodo-piel (potencial de media celda), un filtro pasa-bajas configurable que delimita la banda de interés entre 20 y 500 Hz, y un circuito Right Leg Drive que reinyecta activamente el ruido común al cuerpo a través del electrodo de referencia, usando el propio cuerpo como filtro activo.

Parámetro	Valor / Descripción
Chip	AD8232 — Analog Devices
Alimentación	3.3 V — compatible directo con ESP32
Salida	Analógica 0 – 3.3 V
Interfaz electrodos	Jack 3.5 mm + conectores snap RL / LA / RA
Pines utilizados	OUTPUT → GPIO36 LO- → GPIO34 LO+ → GPIO35
Banda de paso	20 – 500 Hz (con componentes externos del módulo)

3.2 ESP32 — Microcontrolador

Se seleccionó el ESP32 en lugar del Arduino UNO originalmente utilizado en las pruebas iniciales del proyecto. El cambio se justificó por tres razones técnicas determinantes para la calidad de la señal adquirida.

La diferencia más importante es la resolución del convertidor analógico-digital (ADC): el ESP32 dispone de 12 bits, lo que equivale a 4096 niveles de cuantización, frente a los 10 bits (1024 niveles) del Arduino UNO. Esta diferencia cuadruplica la resolución de la señal digitalizada, permitiendo capturar variaciones de amplitud cuatro veces más pequeñas, lo que es fundamental para distinguir gestos que difieren en intensidad sutil.

Parámetro	Valor / Descripción
ADC	12 bits → 4096 niveles (vs. 1024 del Arduino UNO)
Voltaje lógico	3.3 V — compatible directo con AD8232 sin divisor de tensión
Pin analógico	GPIO36 (VP) — pin de solo lectura, menor ruido interno
Comunicación	USB serial a 115200 baud
Procesamiento	Suficiente para ejecutar filtros digitales a 500 Hz en tiempo real

3.3 Electrodo y gel conductor

Se utilizaron electrodos desechables tipo snap, idénticos a los empleados en electrocardiogramas clínicos. Estos electrodos incorporan gel conductor en su superficie adhesiva, aunque durante el desarrollo del proyecto se comprobó que la aplicación de gel adicional antes de cada sesión mejora sustancialmente la calidad de la señal.

El gel conductor cumple una función eléctrica crítica: reduce la impedancia de contacto entre la piel y el electrodo mediante la formación de un puente iónico. Una impedancia de contacto elevada implica que la señal muscular se atenúa antes de llegar al amplificador, resultando en picos de baja amplitud difíciles de distinguir del ruido de fondo.

Protocolo de preparación de la piel (obligatorio antes de cada sesión): limpiar la zona de colocación con alcohol isopropílico. Este procedimiento elimina la grasa natural de la piel y las células muertas superficiales que incrementan la impedancia de contacto. Su aplicación sistemática fue uno de los factores que mayor impacto tuvo en la calidad de las señales obtenidas durante el proyecto.

3.4 Diagrama de conexiones

AD8232	ESP32	Descripción
GND	GND	Tierra común
3.3V	3.3V	Alimentación del módulo
OUTPUT	GPIO36 (VP)	Señal EMG analógica procesada

LO-	GPIO34	Detección de electrodo suelto (-)
LO+	GPIO35	Detección de electrodo suelto (+)
SDN	—	No conectado — chip siempre activo

4. Firmware del ESP32 — Adquisición y preprocesamiento

El firmware desarrollado para el ESP32 implementa cinco bloques funcionales que trabajan en cadena para convertir la señal cruda del ADC en datos limpios, calibrados y listos para el clasificador. A continuación se describe cada bloque y su justificación técnica, seguido del código completo.

4.1 Bloque 1 — Muestreo exacto a 500 Hz

En lugar de emplear la función `delay()`, que bloquea el procesador e introduce errores de temporización acumulativos, se utiliza la función `micros()` para medir el tiempo transcurrido entre muestras. De esta forma se garantiza que el intervalo entre cada lectura sea exactamente de 2000 microsegundos, independientemente del tiempo de ejecución del resto del código.

4.2 Bloque 2 — Promedio móvil de 8 muestras

Se implementa un filtro pasa-bajas digital mediante un buffer circular de 8 muestras (equivalente a 16 ms de señal). Cada nueva muestra reemplaza a la más antigua, y el valor filtrado es el promedio aritmético de las 8 muestras almacenadas. Este filtro suaviza el ruido de alta frecuencia sin introducir latencia perceptible ni distorsionar la forma temporal del gesto.

4.3 Bloque 3 — Calibración automática de línea base

Durante el primer segundo de funcionamiento (500 muestras a 500 Hz), el sistema acumula el valor promedio de la señal en condición de reposo y lo almacena como baseline. Al restar este valor de cada muestra posterior, la señal queda centrada en cero. Esta calibración automática compensa las diferencias entre sesiones: distintos electrodos, distintas posiciones del brazo y variaciones de temperatura de la piel.

4.4 Bloque 4 — Detección de electrodos sueltos

Los pines LO+ y LO- del AD8232 se monitorizan en cada ciclo del loop. Si algún electrodo pierde contacto con la piel, ambos pines toman valor lógico alto. En ese caso, el firmware envía ceros al puerto serial para no interrumpir la línea de tiempo en Teleplot, e imprime una advertencia textual. Cuando el contacto se restablece, la señal se recupera en el ciclo siguiente.

4.5 Bloque 5 — Envolvente IIR

La envolvente se calcula como un filtro IIR (respuesta infinita al impulso) aplicado sobre el valor absoluto de la señal centrada. El parámetro ALPHA controla la velocidad de respuesta: valores altos producen una envolvente más reactiva pero más ruidosa; valores bajos la suavizan pero introducen latencia. El valor $\text{ALPHA} = 0.05$ fue seleccionado experimentalmente como balance óptimo para la visualización en Teleplot.

4.6 Código completo

El siguiente código es el firmware definitivo cargado en el ESP32. Debe subirse una única vez desde Arduino IDE y permanece activo en el microcontrolador para todos los pasos del pipeline.

```
Archivo: firmware_emg_esp32.ino
// EMG - ESP32 | Visualizacion con Teleplot
// Pines: EMG -> GPIO36 | LO+ -> GPIO35 | LO- -> GPIO34

const int EMG_PIN = 36;
const int LO_PLUS = 35;
const int LO_MINUS = 34;

const int SAMPLE_RATE = 500;
const int INTERVAL_US = 1000000 / SAMPLE_RATE;

const int MUESTRAS = 8;
int buf[MUESTRAS]; int idx = 0; long suma = 0;

int promedioMovil(int nuevo) {
    suma -= buf[idx]; buf[idx] = nuevo; suma += nuevo;
    idx = (idx + 1) % MUESTRAS; return suma / MUESTRAS;
}

float envelope = 0;
const float ALPHA = 0.05;
float calcEnvelope(int c) {
    float a = abs(c);
    envelope = ALPHA*a + (1.0-ALPHA)*envelope; return envelope;
}

long baselineSum=0; int baselineCount=0;
int baseline=2048; bool calibrado=false;
const int CALIB_N = 500;
bool ultimoSuelto = false;

void setup() {
    Serial.begin(115200);
    pinMode(LO_PLUS, INPUT); pinMode(LO_MINUS, INPUT);
    analogReadResolution(12);
    analogSetAttenuation(ADC_11db);
    Serial.println("ESP32 EMG listo. Calibrando 1 segundo...");
}

void loop() {
    static unsigned long lastTime = 0;
```

```

unsigned long now = micros();
if (now - lastTime < INTERVAL_US) return;
lastTime = now;

bool suelto = (digitalRead(LO_PLUS)||digitalRead(LO_MINUS));
if (suelto) {
    if (!ultimoSuelto) { Serial.println("ELECTRODO_SUELTO"); ultimoSuelto=true; }
    Serial.println(">EMG:0"); Serial.println(">Envolvente:0"); return;
}
ultimoSuelto = false;

int raw = analogRead(EMG_PIN);
int suavizado = promedioMovil(raw);

if (!calibrado) {
    baselineSum += suavizado; baselineCount++;
    if (baselineCount >= CALIB_N) {
        baseline = (int)(baselineSum / CALIB_N);
        calibrado = true;
        Serial.print("Baseline calibrado: "); Serial.println(baseline);
    }
    Serial.println(">EMG:0"); Serial.println(">Envolvente:0"); return;
}

int centrado = suavizado - baseline;
float env = calcEnvelope(centrado);
Serial.print(">EMG:"); Serial.println(centrado);
Serial.print(">Envolvente:"); Serial.println((int)env);
}

```

5. Pipeline de clasificación en Python

El sistema de clasificación sigue un pipeline de cuatro etapas que transforma la señal cruda del ESP32 en una predicción de gesto en tiempo real. Todas las etapas se ejecutan desde la misma carpeta del proyecto. La instalación de dependencias se realiza una única vez con el siguiente comando:

Instalación de dependencias

```
python -m pip install pyserial pandas numpy scikit-learn websockets
```

5.1 Etapa 1 — Recolección de datos (recolectar.py)

Este script establece la conexión serial con el ESP32 y graba ventanas de señal EMG etiquetadas por gesto. Para cada gesto se registran 20 repeticiones de 2 segundos de duración, lo que resulta en 1000 puntos por ventana ($500 \text{ Hz} \times 2 \text{ s}$). Los datos se almacenan en archivos CSV independientes por gesto, dentro de una subcarpeta denominada datos/.

Procedimiento de ejecución

1. Verificar en el Administrador de dispositivos de Windows el número de puerto COM asignado al ESP32 (ej. COM5, COM7).
2. Modificar la variable PUERTO en el script con el número de puerto correcto.
3. Cerrar el Serial Monitor o Serial Plotter de Arduino IDE si se encuentran abiertos.
4. Ejecutar el script desde la terminal con: `python recolectar.py`
5. Para cada gesto: escribir el número correspondiente, preparar la posición y presionar Enter para iniciar la grabación de 2 segundos.
6. Repetir 20 veces por gesto y escribir q al finalizar.

Archivo: recolectar.py

```
import serial, csv, time, os

PUERTO='COM7'; BAUDRATE=115200; VENTANA=2.0; MUESTRAS=20; FS=500
GESTOS = {'1':'reposo','2':'mano_abierta','3':'giro','4':'puno'}

os.makedirs('datos', exist_ok=True)
ser = serial.Serial(PUERTO, BAUDRATE, timeout=1)
time.sleep(2)
print('=== Recolector EMG ===')
print('1=reposo | 2=mano_abierta | 3=giro | 4=puno | q=salir')

while True:
    tecla = input('Gesto a grabar: ').strip()
    if tecla == 'q': break
    if tecla not in GESTOS: print('Tecla invalida'); continue
```

```

gesto = GESTOS[tecla]
archivo = f'datos/{gesto}.csv'
modo = 'a' if os.path.exists(archivo) else 'w'
with open(archivo, modo, newline='') as f:
    writer = csv.writer(f)
    if modo == 'w':
        writer.writerow(['gesto']+['f{i}' for i in range(int(FS*VENTANA))])
    for rep in range(1, MUESTRAS+1):
        input(f' [{rep}/{MUESTRAS}] Prepara gesto -> ENTER')
        muestras=[]; inicio=time.time(); ser.flushInput()
        while time.time()-inicio < VENTANA:
            l=ser.readline().decode('utf-8',errors='ignore').strip()
            if l.startswith('>EMG:'):
                try: muestras.append(int(l.split(':')[1]))
                except: pass
        n=int(FS*VENTANA)
        muestras=muestras[:n] if len(muestras)>=n else muestras+[0]*(n-len(muestras))
        writer.writerow([gesto]+muestras)
        print(f' OK ({len([x for x in muestras if x!=0])} puntos)')
ser.close()

```

5.2 Etapa 2 — Extracción de características (features.py)

Las 1000 muestras de cada ventana de señal no pueden ser entregadas directamente al clasificador, ya que varían en escala y no son comparables entre sí de forma directa. En su lugar, se extraen seis descriptores numéricos que resumen las propiedades estadísticas y energéticas de la señal. Este proceso se denomina extracción de características o feature engineering.

Sigla	Nombre completo	Qué mide
MAV	Mean Absolute Value	Amplitud promedio — distingue reposo (bajo) de contracciones fuertes (alto).
RMS	Root Mean Square	Energía de la señal — correlaciona con la fuerza de contracción muscular.
ZCR	Zero Crossing Rate	Frecuencia de cruce por cero — refleja la frecuencia dominante del gesto.
VAR	Variance	Dispersión — alta durante contracción activa, baja en reposo.
WL	Waveform Length	Longitud de la forma de onda — mide la complejidad y actividad acumulada.

Archivo: features.py

```
import numpy as np, pandas as pd, glob

def extraer_features(v):
    v = np.array(v, dtype=float)
    return [np.mean(np.abs(v)),      # MAV
            np.sqrt(np.mean(v**2)),  # RMS
            np.sum(np.diff(np.sign(v))!=0), # ZCR
            np.var(v),               # VAR
            np.sum(np.abs(np.diff(v))), # WL
            np.sum(np.abs(v))]       # IEMG

filas=[]
for archivo in glob.glob('datos/*.csv'):
    df=pd.read_csv(archivo)
    for _,fila in df.iterrows():
        features=extraer_features(fila.iloc[1:].values.astype(float))
        filas.append([fila['gesto']]+features)

cols=['gesto','MAV','RMS','ZCR','VAR','WL','IEMG']
dataset=pd.DataFrame(filas,columns=cols)
dataset.to_csv('dataset.csv',index=False)
print(f'Dataset: {len(dataset)} muestras')
print(dataset.groupby('gesto').size())
```

3.4 Por dentro: ¿qué hace features.py exactamente?

Para que esto no se quede solo en “ejecute el script y ya”, vale la pena entender qué pasa por dentro. Cuando se graba un gesto, lo que realmente le llega a Python es una ventana de 1000 números seguidos (los 500 Hz por los 2 segundos de grabación de los que se habló arriba). Eso es la señal cruda: un montón de valores que suben y bajan, con ruido incluido. El problema es que un clasificador no puede aprender directamente de 1000 números que cambian de forma cada vez — necesita un resumen fijo y consistente de cada ventana. Eso es justo lo que hace features.py: agarra esos 1000 valores y los convierte en solo 6 números, las características de la tabla de arriba.

En la práctica, el cálculo de cada característica se ve más o menos así, usando NumPy:

features.py — cálculo de las 6 características (simplificado)

```
import numpy as np
```

```
def extraer_features(ventana):
    ventana = np.array(ventana)

    mav = np.mean(np.abs(ventana))          # MAV
    rms = np.sqrt(np.mean(ventana**2))      # RMS
    zcr = np.sum(np.diff(np.sign(ventana)) != 0) # ZCR
    var = np.var(ventana)                   # VAR
    wl = np.sum(np.abs(np.diff(ventana)))    # WL
    iemg = np.sum(np.abs(ventana))           # IEMG

    return [mav, rms, zcr, var, wl, iemg]
```

Cada una mide algo distinto de la señal, por eso se complementan entre sí: MAV y RMS miden qué tan fuerte está la contracción, ZCR mide qué tan rápido oscila la señal (algo así como su frecuencia), VAR mide qué tan dispersos están los valores respecto al promedio, WL suma todos los saltos entre un valor y el siguiente (entre más brusca la señal, más alto sale), e IEMG es la activación acumulada de toda la ventana. El script repite este cálculo para cada repetición grabada en cada archivo de datos/ (reposo.csv, puno.csv, mano_abierta.csv, giro.csv), le pega la etiqueta del gesto correspondiente a cada fila, y junta todo con pandas en dataset.csv. Al final, cada fila del CSV queda así: [MAV, RMS, ZCR, VAR, WL, IEMG, gesto] — y eso es literalmente lo único que el clasificador del Paso 3 va a ver.

Para que se entienda la magnitud: con 6 características, cada muestra grabada es un punto en un espacio de 6 dimensiones. No se puede dibujar en una hoja, pero la idea de “separar grupos de puntos” es exactamente la misma que se usa en el Paso 3 para entrenar el clasificador.

5.3 Etapa 3 — Entrenamiento del clasificador (entrenar.py)

Se seleccionó un clasificador SVM (Support Vector Machine) con kernel RBF (Radial Basis Function) por tres razones fundamentales: excelente rendimiento con conjuntos de datos pequeños (menos de 200 muestras), capacidad de separar clases no linealmente separables en el espacio de características, y baja latencia de inferencia adecuada para predicción en tiempo real.

El pipeline de entrenamiento aplica los siguientes pasos en secuencia: división del dataset en 80% entrenamiento y 20% prueba con estratificación de clases, normalización mediante StandardScaler (media 0, desviación estándar 1), entrenamiento del SVM con C=10 y gamma='scale', y evaluación con reporte de clasificación por clase y matriz de confusión.

Archivo: entrenar.py

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import pickle

df=pd.read_csv('dataset.csv')
X=df[['MAV','RMS','ZCR','VAR','WL','IEMG']].values
y=df['gesto'].values

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42,stratify=y)

scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

modelo=SVC(kernel='rbf',C=10,gamma='scale')
modelo.fit(X_train,y_train)

y_pred=modelo.predict(X_test)
print(f'Precision: {(y_pred==y_test).mean()*100:.1f}%')
print(classification_report(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))

pickle.dump(modelo,open('modelo.pkl','wb'))
pickle.dump(scaler,open('scaler.pkl','wb'))
print('Modelo guardado: modelo.pkl')
```

Resultado obtenido: precisión global del 94.1%. Mano abierta y reposo con clasificación perfecta (100%). Puño con 100% de recall. Giro con 75% de recall — una muestra fue confundida con puño, lo cual es esperable dada la similitud muscular entre ambos gestos. Con 30 o más muestras por gesto la precisión mejoraría significativamente.

5.4 Etapa 4 — Predicción en consola (predecir.py)

Una vez entrenado el modelo, este script permite verificar el funcionamiento del clasificador en tiempo real directamente en la terminal, sin necesidad de abrir el juego. Lee el puerto serial, acumula 1000 muestras en un buffer deslizante, extrae los seis descriptores, normaliza con el scaler guardado, y muestra el gesto predicho actualizando continuamente la misma línea de la terminal.

```
Uso: predecir.py  
python predecir.py  
  
# Salida esperada en terminal:  
-> REPOSO  
-> PUNO  
-> MANO ABIERTA  
-> GIRO
```

4.4 Por dentro: ¿cómo aprende el SVM a reconocer los gestos?

Esta es la parte que seguramente preguntan en la sustentación, así que vale la pena tenerla clara y poder explicarla sin enredarse. Cada muestra del dataset es un punto en un espacio de 6 dimensiones (una por cada característica: MAV, RMS, ZCR, VAR, WL, IEMG). Un SVM (Support Vector Machine, o “máquina de vectores de soporte”) busca la mejor frontera posible para separar esos puntos según el gesto al que pertenecen — algo parecido a trazar líneas que dividan grupos de puntos de colores distintos en una hoja de papel, pero en 6 dimensiones en lugar de 2.

El detalle está en que las señales musculares casi nunca se dejan separar con líneas rectas: los grupos se enredan entre sí. Ahí entra el kernel RBF (Radial Basis Function): en lugar de forzar una frontera recta, lo que hace es “doblar” el espacio para poder dibujar fronteras curvas alrededor de cada grupo de puntos. Por eso la guía dice que es “ideal para clases no linealmente separables” — es justo lo que pasa con los gestos EMG, donde reposo, puño, mano abierta y giro no quedan ordenados en línea recta cuando se miran sus 6 características.

En código, entrenar.py hace algo parecido a esto:

```
entrenar.py — entrenamiento del clasificador (simplificado)  
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.svm import SVC
from sklearn.metrics import classification_report
import pickle

# 1. Cargar el dataset que armo features.py
datos = pd.read_csv("dataset.csv")
X = datos[["MAV", "RMS", "ZCR", "VAR", "WL", "IEMG"]]
y = datos["gesto"]

# 2. Normalizar: el SVM es sensible a la escala de los numeros
scaler = StandardScaler()
X_norm = scaler.fit_transform(X)

# 3. Separar datos de entrenamiento y de prueba
X_train, X_test, y_train, y_test = train_test_split(
    X_norm, y, test_size=0.2, stratify=y, random_state=42
)

# 4. Crear y entrenar el modelo
modelo = SVC(kernel="rbf", C=10, gamma="scale")
modelo.fit(X_train, y_train)

# 5. Revisar que tan bien le fue
predicciones = modelo.predict(X_test)
print(classification_report(y_test, predicciones))

# 6. Guardar el modelo y el normalizador
pickle.dump(modelo, open("modelo.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))

```

Algunas cosas de ese código que vale la pena tener claras para explicarlas con seguridad:

- **StandardScaler (el normalizador):** deja todas las características en la misma escala (promedio 0, desviación 1). Sin esto, una feature con números grandes como IEMG podría “pesar” más que una con números pequeños como ZCR, solo por la escala y no porque sea más importante. Por eso scaler.pkl se guarda junto al modelo: cualquier dato nuevo que se quiera predecir debe normalizarse exactamente igual.
- **train_test_split con test_size=0.2:** separa el 80% de las muestras para entrenar el modelo y deja un 20% “escondido” para examinarlo después. Así se puede comprobar si el modelo realmente aprendió a generalizar o si solo memorizó los datos de entrenamiento.
- **C y gamma:** son los parámetros que controlan qué tan flexible es la frontera curva del kernel RBF. C controla qué tan estricto es el modelo con los errores durante el entrenamiento, y gamma qué tan “ajustadas” quedan las curvas

alrededor de cada grupo de puntos. No hay una fórmula mágica para elegirlos: se prueban distintas combinaciones y se deja la que mejor resultado dé.

- **classification_report:** es exactamente la tabla que aparece en consola con precision, recall y f1-score por gesto — de ahí sale el 94.1% de precisión general que se reporta en el Paso 3.

En resumen, el modelo no hace nada misterioso: con los 81 ejemplos grabados aprendió en qué zona del espacio de 6 dimensiones suele caer cada gesto, y cuando llega un dato nuevo simplemente revisa de qué lado de esas fronteras curvas queda para decidir la etiqueta.

En criollo: el SVM no “ve” gestos ni señales musculares — ve 6 números por muestra y aprende patrones geométricos entre ellos. Por eso la calidad de las características (Paso 2) y de los datos grabados (Paso 1) importa tanto como el algoritmo en sí.

6. Interfaz gráfica — Juego del dinosaurio controlado por EMG

Como aplicación demostrativa del sistema se desarrolló un juego del dinosaurio controlado en tiempo real por señales musculares. La interfaz fue diseñada para mostrar simultáneamente el juego en pantalla completa y un panel de monitoreo lateral con el gesto detectado, la señal EMG en vivo y el estado de conexión con el ESP32.

6.1 Arquitectura del sistema

La comunicación entre el hardware y la interfaz web se realiza mediante dos protocolos complementarios que operan sobre el mismo computador:

Componente	Puerto	Función
ESP32	USB / Serial	Lee la señal muscular y la transmite al computador a 500 Hz.
servidor.py	WebSocket :8765	Recibe la señal, predice el gesto y lo transmite al juego.
http.server	HTTP :8080	Sirve el archivo juego.html al navegador web.
juego.html	Navegador	Recibe el gesto por WebSocket y controla el dinosaurio.

6.2 ¿Por qué se necesita el servidor local HTTP?

Una pregunta frecuente es por qué no se puede abrir juego.html directamente con doble clic. La razón es una restricción de seguridad implementada por todos los navegadores modernos: cuando un archivo HTML se carga mediante el protocolo file://, el navegador bloquea las conexiones WebSocket salientes por política de origen cruzado. Esto impide que el juego se comunique con servidor.py.

La solución consiste en ejecutar un servidor HTTP local con el comando `python -m http.server 8080`, lo que sirve el archivo bajo el protocolo `http://localhost`, que el navegador sí permite conectar mediante WebSocket. La diferencia es fundamental:

```
file:///C:/Users/.../juego.html  <- BLOQUEADO por el navegador
http://localhost:8080/juego.html  <- PERMITIDO — WebSocket funciona
```

Regla de oro: siempre acceda al juego desde la URL `http://localhost:8080/juego.html` en el navegador. Nunca haciendo doble clic en el archivo.

6.3 Mapeo de gestos a acciones del juego

Ico no	Gesto	Acción en el juego
	Reposo	El dinosaurio corre normalmente. En este estado no hay colisiones — es el estado seguro.
	Puño cerrado	El dinosaurio salta para esquivar los cactus del suelo.
	Mano abierta	El dinosaurio se agacha para esquivar los pterodáctilos aéreos.
	Giro de muñeca	Modo turbo — la velocidad aumenta $\times 1.8$. Mayor puntuación por tiempo.

6.4 Código del servidor (servidor.py)

El servidor utiliza `asyncio` para manejar concurrentemente la lectura del puerto serial y las conexiones `WebSocket`. Una mejora clave respecto a versiones anteriores es el uso de una ventana deslizante: en lugar de esperar 2 segundos completos para actualizar el gesto, el sistema predice cada 0.5 segundos usando los últimos 2 segundos de datos. Esto reduce la latencia percibida de 2 segundos a 0.5 segundos sin sacrificar la calidad de los descriptores. Adicionalmente, el servidor escucha en 0.0.0.0 en lugar de localhost para garantizar compatibilidad con todos los navegadores.

Archivo: `servidor.py`

```
import asyncio, websockets, serial, numpy as np
import pickle, json
from collections import deque

PUERTO='COM7'; BAUDRATE=115200; FS=500
VENTANA=2.0; N=int(FS*VENTANA); WS_PORT=8765

modelo=pickle.load(open('modelo.pkl','rb'))
scaler=pickle.load(open('scaler.pkl','rb'))
clientes=set(); gesto_actual='reposo'
```

```

async def broadcast(msg):
    if clientes:
        await asyncio.gather(*[c.send(msg) for c in clientes])

async def handler(websocket):
    clientes.add(websocket)
    try: await websocket.send(json.dumps({'gesto': gesto_actual}))
    except: pass
    try: await websocket.wait_closed()
    finally: clientes.discard(websocket)

async def leer_serial():
    global gesto_actual
    ser=serial.Serial(PUERTO,BAUDRATE,timeout=1)
    await asyncio.sleep(2)
    ser.flushInput()
    buffer=deque(maxlen=N); frameCount=0
    while True:
        try:
            l=ser.readline().decode('utf-8',errors='ignore').strip()
            if not l.startswith('>EMG:'): continue
            buffer.append(int(l.split(':')[1]))
            if len(buffer)==N and frameCount%(N//4)==0:
                b=np.array(list(buffer),float)
                f=[np.mean(np.abs(b)),np.sqrt(np.mean(b**2)),
                    np.sum(np.diff(np.sign(b))!=0),np.var(b),
                    np.sum(np.abs(np.diff(b))),np.sum(np.abs(b))]
                gesto=modelo.predict(scaler.transform([f]))[0]
                gesto_actual=gesto
                await broadcast(json.dumps({'gesto': gesto}))
                print(f'-> {gesto}')
            frameCount+=1
        except: await asyncio.sleep(0.01)

async def main():
    server=await websockets.serve(handler,'0.0.0.0',WS_PORT)
    print(f'WebSocket activo en ws://localhost:{WS_PORT}')
    await asyncio.gather(server.wait_closed(),leer_serial())

asyncio.run(main())

```

6.5 Procedimiento completo de ejecución del juego

Para ejecutar el juego se requieren exactamente dos terminales abiertas de forma simultánea, además del navegador web. El procedimiento es el siguiente:

Terminal 1 — Servidor de gestos

Abrir una terminal en la carpeta del proyecto y ejecutar:

```
python servidor.py
```

La terminal debe mostrar el siguiente mensaje y permanecer activa:

```
WebSocket activo en ws://localhost:8765  
Conectando a COM7...  
-> reposo  
-> puno  
...
```

Esta terminal no debe cerrarse mientras el juego esté en uso.

Terminal 2 — Servidor del juego (HTTP local)

Abrir una segunda terminal en la misma carpeta y ejecutar:

```
python -m http.server 8080
```

La terminal debe mostrar:

```
Serving HTTP on 0.0.0.0 port 8080 ...
```

Esta terminal tampoco debe cerrarse mientras el juego esté en uso.

Navegador — Acceso al juego

Abrir Chrome o Edge y escribir la siguiente URL en la barra de direcciones:

```
http://localhost:8080/juego.html
```

Si la configuración es correcta, el indicador de conexión en la esquina superior del juego se mostrará en verde con el texto Conectado · ESP32 activo. A partir de ese momento, los gestos musculares controlan el dinosaurio en tiempo real.

7. Resultados obtenidos

7.1 Precisión del clasificador

El modelo SVM fue entrenado con 81 muestras (20 por gesto, excepto puño con 21) y evaluado sobre un conjunto de prueba de 17 muestras, obteniendo una precisión global del 94.1%.

Gesto	Precisión	Recall	F1-Score	Resultado
mano_abierta	1.00	1.00	1.00	Clasificación perfecta
reposito	1.00	1.00	1.00	Clasificación perfecta
puño	0.83	1.00	0.91	Sin falsos negativos
giro	1.00	0.75	0.86	1 muestra confundida con puño

7.2 Análisis de resultados

El único error de clasificación fue una muestra del gesto giro que fue asignada a la clase puño. Este comportamiento es esperable: ambos gestos implican contracción de los músculos del antebrazo, y sus descriptores de energía (MAV y RMS) presentan valores similares. La diferencia principal reside en el ZCR (tasa de cruce por cero), que es mayor en el giro por la activación diferencial de los músculos supinadores. Con un mayor número de muestras de entrenamiento para el gesto giro, el clasificador aprendería a separar estos casos límite con mayor eficacia.

7.3 Lecciones aprendidas

- El hardware físico es tan determinante como el algoritmo de clasificación: un electrodo mal adherido o la ausencia de gel conductor pueden arruinar completamente la calidad de la señal, independientemente de la sofisticación del software.

- El cambio de Arduino UNO a ESP32 fue decisivo: la resolución de 10 bits del Arduino generaba una cuantización excesiva que dificultaba la separación entre clases. Los 12 bits del ESP32 resolvieron este problema.
- La calibración automática de línea base eliminó la necesidad de recalibración manual entre sesiones, haciendo el sistema robusto a cambios de posición, temperatura y tipo de electrodo.
- Teleplot resultó ser significativamente superior al Serial Plotter de Arduino como herramienta de visualización, al permitir múltiples variables simultáneas y comportarse como un osciloscopio digital.
- El uso de ventana deslizante en servidor.py redujo la latencia de detección de gestos de 2 segundos a 0.5 segundos, mejorando sustancialmente la experiencia de control en el juego.

8. Trabajo futuro

Los siguientes desarrollos están identificados como próximos pasos para la evolución del sistema:

- OE3 — Características en dominio de frecuencia: incorporar descriptores espectrales (potencia por banda, frecuencia media) para mejorar la separabilidad entre gestos con perfiles energéticos similares, como giro y puño.
- OE4 — Comparación de clasificadores: evaluar Random Forest y kNN junto al SVM para determinar el balance óptimo entre precisión e inferencia en tiempo real.
- Mayor volumen de datos: incrementar el número de repeticiones por gesto a 50 o más, idealmente con múltiples participantes, para construir un clasificador generalizable más allá del usuario entrenado.
- OE5 — Interfaz web completa: desarrollar la página web definitiva con diseño tipo dashboard médico-gaming, integrando la visualización de la señal EMG en vivo, el gesto detectado con animaciones y el control del juego.
- Transmisión inalámbrica: reemplazar la conexión USB por Bluetooth BLE para eliminar el cable entre el usuario y el computador durante las sesiones de medición.

9. Herramientas y referencias

Hardware

- Sensor AD8232 — Analog Devices. Datasheet rev. D. Analog Devices, Inc.
- Microcontrolador ESP32 — Espressif Systems. Technical Reference Manual.

- Electroodos desechables tipo snap para ECG/EMG.

Software

- Arduino IDE 1.8.7 — desarrollo del firmware del ESP32.
- Teleplot (extensión Visual Studio Code) — visualización serial tipo osciloscopio.
- Python 3.14 — librerías: pyserial, numpy, pandas, scikit-learn, websockets.
- scikit-learn — Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR.

Referencias académicas

- Phinyomark, A. et al. (2012). Feature reduction and selection for EMG signal classification. Expert Systems with Applications, 39(8), 7420-7431.
- Oskoei, M.A., Hu, H. (2007). Myoelectric control systems — A survey. Biomedical Signal Processing and Control, 2(4), 275-294.